

React Js

Features:-

- OpenSource
- FrontEnd JS Library
- Creating Component based User Interface
- Developed by META
- SinglePage Application(Website may contain multiple pages but they doesn't reload)
- Mobile Friendly Application
- Less Code
- Can create Interactive User Interface
- Reusability(to create reusable UI Components and manage the state of those components)
- Scalability(can create large real time application)
- websites powered by React
 - Facebook, popular social media application
 - Instagram, popular photo sharing application
 - Netflix, popular media streaming application
 - Code Academy, popular online training application
 - Reddit, popular content sharing application

Note:

React → collection of components → as functional components(class components also)

Imperative vs Declarative Programming:-

Imperative

```
<script type="text/javascript">
const app = document.getElementById('app');
const header = document.createElement('h1');
const headerContent = document.createTextNode('Develop. Preview. Ship. 🚀');
header.appendChild(headerContent);
app.appendChild(header);
</script>
```

Declarative

```
<script type="text/jsx">
const app = document.getElementById("app")
ReactDOM.render(<h1>Develop. Preview. Ship. 🚀</h1>, app)
</script>
```

Note:

Imperative programming is like giving a chef step-by-step instructions on how to make a pizza. Declarative programming is like ordering a pizza without being concerned about the steps it takes to make the pizza.

Installation:-

Download & Install NODE from nodejs.org website

Install VS Code

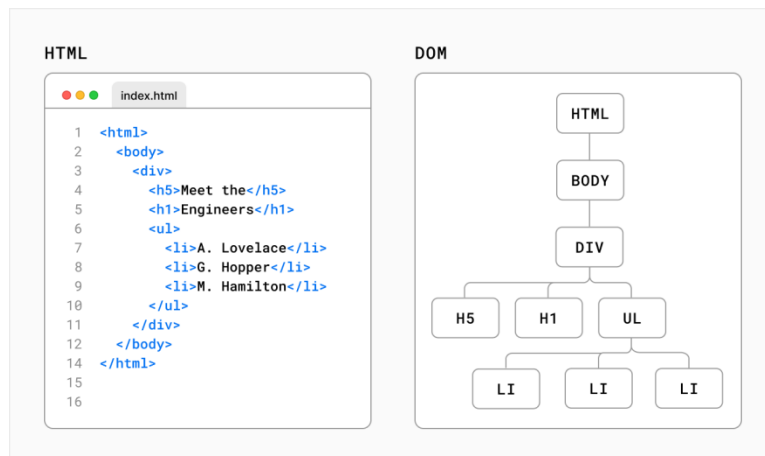
go to extension → es7 rACT → ES7+REACT/REDUX/REACT-NATIVE...

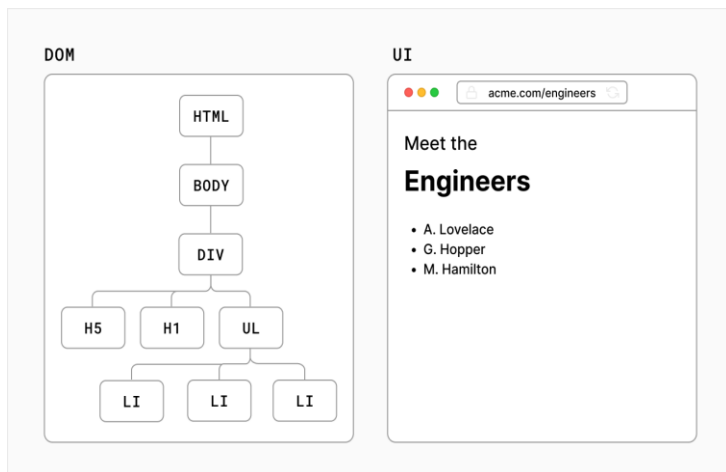
Note:

- npm: Node Package Manager
- npx: Node Package Execute (npm package runner)
- To execute any JavaScript package available on the NPM registry without even install it

DOM vs VDOM:-**DOM: Document Object Model**

- HTML document as a tree structure where in each node is an object representing a part of the document
- The DOM represents a document with a logical tree
- The browser reads the HTML and constructs the Document Object Model (DOM).





VDOM:Virtual DOM

- React generates a virtual DOM, which is an in-memory data-structure cache. Only the final DOM updates have been updated in the browser's DOM.
- A programming concept where an ideal or “virtual” representation of a UI kept in memory and synced with the “real” DOM by a library such as ReactDOM. This process is called Reconciliation

Eg:

Place `<div id="one">....</div>` into App.js after header tag → run → view source → we can't see `<ul>` tag

Note:

index.js

- `const root=ReactDOM.createRoot(....root)`
Injecting our JSX inside root element using react dom

Create React Project:-

- In vscode → open folder → create folder → type project name(MyReactApp) → select folder
- View → terminal →

```
npm create vite@latest myreactapp
```

 - ✓ name must be in lower case

```
choose react (use downarrow key and press enter)
```

```
choose javascript
```

```
cd myreactapp
```

```
npm install
```

 - ✓ to create react app with supported file system automatically

Note:

If can't execute command → open powershell → `Set-ExecutionPolicy RemoteSigned -Scope CurrentUser`

To run:-

```
npm run dev
```

ctrl+c - to stop running

Components:-

- similar to pure javascript functions
- splits the user interface into independent parts, each of which can be processed separately
- to simplify the code by separating the functionality into reusable independent code
- Components can be used as functions and as classes.
- Components also have a state, and props

Functional Components:-

- each component must be in separate file
- App.jsx - root component
- all component name must be in CamelCase

To insert function:

type rfce → to insert function

```
import React from 'react'
function ComponentName() {
  return (
    <div> ComponentName </div>
  )
}
export default ComponentName
```

type rafce --> to insert arrow function

```
import React from 'react'
const ComponentName = () => {
  return (
    <div> ComponentName </div>
  )
}
export default ComponentName
```

Note: Create JSX file(Component) and type rfce → enter → to auto generate code

JSX: Javascript Expressions:-

JSX is a syntax extension for JavaScript that allows to describe UI in a familiar HTML-like syntax.

Note:

Browsers don't understand JSX out of the box, so need a JavaScript compiler, such as a [Babel](#), to transform your JSX code into regular JavaScript.

App.js file:

```
function App{
  return(
    {JSX code}{known as declarative programming}
  )
}
```

Note:

JSX must have one parent element

```
return(
```

```
<div> <- any tag or <></> (fragment)
      -code-
</div> )
```

React JS Disadvantages:-

- 1.code is written in JSX(HTML and CSS are part of the javascript code) ,other frameworks keep HTML separate from the javascript code.
- 2.huge file size.

Note:

Elements Must be Closed

```
<input type="text"/>
```

Use attribute className instead of class in JSX:

```
<h1 className="myclass">Hello World</h1>
```

Eg: Simple html tags

App.js:

```
function App(){
  return (
    <div>
      <h1>Hello ReactJs!</h1>
    </div>
  );
}
export default App;
```

Eg: jsx code

App.js:

```
function App(){
  let a=10
  return (
    <div>
      <h1>Hello ReactJs!</h1>
      <br/>
      {a}
    </div>
  );
}
export default App;
```

} html code
→ jsx code

Eg: jsx code

App.js:

```
function App(){
  let a=10
```

```

        let b=20
        <h1>Hello ReactJs!</h1>
return (
  <div>
    {b}
    <br/>
    {a}
  </div>
);
}
export default App;

```

Eg:

App.js:

```

function App(){
  let a=10
  function show(){
    alert("Hello")
  }
  show()
return(
  <div>
    <h1>Hello ReactJs!</h1>
    <br/>
    {a}
  </div>
);
}
export default App;

```

Eg:

App.js:

```

function App(){
  let a=10
  function show(){
    alert("Hello")
  }
return(
  <div>
    <h1>Hello ReactJs!</h1>
    <br/>

```

```

        {a}
        {show()}
    </div>
  );
}
export default App;

```

Eg: functionalcomponent

→ Right click on src folder → file → MyButton.jsx

MyButton.jsx

```

function MyButton(){
  return(
    <button>I'm a button</button>
  );
}
Export default MyButton;

```

App.js:

```

function App(){
  let a=10
  function show(){
    alert("Hello")
  }
  show()
  return(
    <div>
      <h1>Hello ReactJs!</h1>
      <br/>
      {a}
      <MyButton/>
    </div>
  );
}
export default App;

```

Assessment:

Create component named OrderedList,UnOrderedList,DefinitionList with suitable content and include those into App component

Function call in React

```
import React from 'react'
function Sample3() {
  function show()
  {
    alert("Welcome")
  }
  function display()
  {
    console.log("Hello")
  }
  function put()
  {
    console.log("put")
  }
  show()
  function add(x,y)
  {
    console.log(x+y)
  }
  return (
    <div>
{display()}
call put function whenever button clicked
    <button onClick={put}>PUT</button><br/>
call put function only when page load
    <button onClick={put()}>PUT</button><br/>
call add function only when page load
    <button onClick={add(3,4)}>ADD</button><br/>
call add function whenever button clicked
    <button onClick={()=>add(3,4)}>ADD</button><br/>
a</div>
  )
}
export default Sample3
```

Eg: JS objects

App.js

```
const user = {  
  name: 'Hedy Lamarr',  
  imageUrl: 'https://i.imgur.com/yXOvdOSs.jpg',  
  imageSize: 90,  
};
```

```
<h1>{user.name}</h1>
```

```
<img src={user.imageUrl} alt={'Photo of ' + user.name} width={user.imageSize} height={user.imageSize}/>
```

Assessment:

- Create student object with sno,sname,course,duration attributes and display these line by line
- |||^{ly} for books,account,course

Conditional rendering

Eg: using if

```
const x = 5  
let text  
if(x < 10){  
  text = "Hello"  
}  
<h1>{text}</h1>
```

Eg: using && (logical and) (shorter code)

```
let x = 5  
<h1>{ x < 10 && "Hello" }</h1>
```

Eg: using if else

```
let x = 5  
let y  
if(x < 10){  
  y = "Hello"  
}  
else{  
  y = "Goodbye"  
}  
<h1>{y}</h1>
```

Eg: using ?: (conditional op) (shorter code)

```
let x =5;
<h1>{(x)<10?"Hello":"Goodbye"}</h1>
```

Eg:

```
let content;
let isLoggedIn=true;
if(isLoggedIn){
    content = <AdminPanel/>;
}else{
    content = <LoginForm/>;
}
{content}
```

Eg:

```
let content;
let isLoggedIn=true;
{isLoggedIn ? (<AdminPanel/>) : (<LoginForm/>)}
```

Note:

don't need the else branch → {isLoggedIn &&<AdminPanel />}

Rendering lists

Eg:Using for loop

```
const numbers = [1, 2, 3, 4, 5];
let i;
for(i=0;i<numbers.length;i++)
    console.log(numbers[i])
```

Eg:Using map function

```
const numbers = [1, 2, 3, 4, 5];
numbers.map((x)=>console.log(x) )
```

Eg:

```
const numbers = [1, 2, 3, 4, 5];
<ul>
    {numbers.map((number) => (
        <li key={number}>{number}</li>
    ))}
</ul>
```

Eg:

Header.js

```
<div>Header</div>
```

Content.js

```
function show()
{
    const names=["Grow","Earn","Give"];
    const nos=Math.floor(Math.random()*3);
    return names[nos];
}
<p>Let's {show()} Money!</p>
```

Footer.js

```
const Year=new Date();
<footer>Copyright &copy; {year.getFullYear()} </footer>
```

App.js

```
import Header from ".Header";
import Content from ".Content";

<div><Header/></div>
<div><Content/></div>
<div><Footer/></div>
```

Note: import statements are automatically inserted when you use the autocomplete option while typing component name

Eg:method:1

```
const products = [
  { title: 'Cabbage', isFruit: false, id: 1 },
  { title: 'Garlic', isFruit: false, id: 2 },
  { title: 'Apple', isFruit: true, id: 3 },
];

const listItems = products.map(product =>
  <li key={product.id} style={{color: product.isFruit ? 'magenta' : 'darkgreen'}}>
    {product.title}
  </li>
);

<ul>{listItems}</ul>
```

Eg:method:2

```
const products = [
  { title: 'Cabbage', isFruit: false, id: 1 },
  { title: 'Garlic', isFruit: false, id: 2 },
  { title: 'Apple', isFruit: true, id: 3 },
];

<ul>
```

```

    {products.map((x)=>
      <li style={x.isFruit ? {color:'magenta'}:{color:'green'}}>
        {x.title}
      </li>
    )
  }
</ul>

```

Note: <https://nextjs.org/learn/foundations/from-javascript-to-react>

Component with CSS Styles:-

Eg: Method1

Header.js

```

<Header style={{backgroundColor:'blue',color:'white'}}>
  <h1>To Do List</h1>
</Header>

```

Note:{{}}→ outer {} is for JSX and inner{} is for css object

Eg:Method2

```

const headerStyle={backgroundColor:'blue',color:'white'}
'<Header style={headerStyle}>

```

Eg:Method3

```

<Header class="MyHeader">.....

```

header.css:

```

.MyHeader
{
  backgroundColor:'blue';
  color:'white';
}

```

Eg:

```

var myStyle = {
  fontSize: 80,
  fontFamily: 'Courier',
  color: '#003300'
}
<h1 style = {myStyle}>www.javatpoint.com</h1>

```

Note:

Invalid jsx cods:

```

{object}      eg:{a:1,b:2}
{boolean}     eg:{1===1}

```

Object and boolean are not react child

Jsx comment: `{/*.....*/}` shift+alt+a

Class Components:

```
Class Show extends React.Component{
  render(){
    return <h1>Hello,{this.props.name}</h1>;
  }
}
```

Function Types

Type 1: function with name function show(){ return 10; } let x=show(); console.log(x);	Type 1: function with name a=[1,2,3,4,5] a.map(function show(b) { console.log(b) })
Type 2: anonymous function let x=function(){ return 10; } console.log(x());	Type 2: anonymous function a.map(function (b) { console.log(b) })
Type 3: arrow function let x=()=> return 10 console.log(x);	Type 3: arrow function a.map(b=> console.log(b)) x=a.map(b=> b*b)

Events

Eg:

```
function MyButton(){
  function handleClick(){
    alert('You clicked me!');
  }
  <button onClick={handleClick}>Click me</button>
```

Eg:

```
function show1{
  console.log('Hello from show1')
}
<button onClick={show1}>Click</button>
```

Eg:

```
const show2={()=>{
  console.log('Hello from show2')
}}
<button onClick={show2()}>Click</button>
```

Note: cant call show2 when button clicked and called automatically when page loaded

Assessment:

Calculate area and perimeter of circle with predefined radius value and display the result in console while click the circle button. Similarly for square,rectangle button

Eg:Use anonymous function in onClick

```
const show2={()=>{
  console.log('Hello from show2')
}}
<button onClick={()=>show2()}>Click</button>
```

Eg:

```
const show3=(x)=>{
  console.log(`Hello from show3 for ${x}`)
}
<button onClick={()=> show3('csc')}>Click</button>
```

Note:

- {handleClick()}→call handleClick function only when page load and reload not when click the button
- {handleClick}→call handleClick function whenever click the button

Assessment:

Calculate area and perimeter of circle by passing radius to function and display the result in console while click the circle button. Similarly for square,rectangle button

Eg:To get event details

```
const show4=(e)=>{
  console.log(e)
}
<button onClick={(e)=> show4(e)}>Click</button>
```

Eg:To get event source(for which the current event raise)

```
const show5=(e)=>{
  console.log(e.target)
}
<button onClick={(e)=> show5(e)}>Click</button>
```

Eg:

```
const show6=(e)=>{
  console.log(e.target.innerText)
```

```

    }
    <button onClick={(e)=> show6(e)}>Click</button>

```

Eg:

```

function show(){
    const names=["Grow","Earn","Give"];
    const nos=Math.floor(Math.random()*3);
    return names[nos];
}
const handleClick=(x)=>{
    console.log(` ${x}!`)
}
<p onDoubleClick={()=>handleClick('Hai')}>Let's {show()} Money!</p>

```

Note:

We cant change any content in a page by calling any function. To change use “props”

Props:

- Properties
- Props are passed from a parent component to a child component.
- Props **cannot be modified** by the child component.
- Components can be reused with different props


```

                <Greeting name="Alice" />
                <Greeting name="Bob" />
            
```
- Useful for sending values, callbacks (functions), or configuration
- can set default values for props using defaultProps


```

                Greeting.defaultProps = {
                    name: 'Guest'
                };
            
```
- Use PropTypes to validate props and improve code robustness:


```

                import PropTypes from 'prop-types';
                Greeting.propTypes = {
                    name: PropTypes.string.isRequired
                };
            
```
- They allow components to be customized with dynamic input.

eg:

```

function show(props){
    return <h1>Hello,{props.name}</h1>;
}

```

Eg:4

Comp1.js

```
function Comp1(props){
  const x=10;
  return(
    <>
      <p>HELLO,{props.zValue}</p>
    </>
  );
}
```

App.js

```
Function App(){
  const z=12+45-4;
  Return(
    <Comp1 zValue={z}/>
  );
}
```

Eg: Props with Image and Text

App.js

```
<ProfileCard name="Emma Watson" description="An actress and activist."
imageUrl="https://via.placeholder.com/150"/>
```

ProfileCard.js

```
function ProfileCard(props) {
  return (
    <div style={{ border: 'px solid #ccc', padding: '10px', width: '250px' }}>
      <img src={props.imageUrl} alt={props.name} style={{ width: '100%' }} />
      <h2>{props.name}</h2>
      <p>{props.description}</p>
    </div>
  );
}
```

Eg:

Index.js

```
<App unicode="2192"/>
```

App.js

```
const App = ({ unicode }) => {
  return(
    <h1>
      This is a Right arrow
      {String.fromCharCode(parseInt(unicode, 16))}
    </h1>
  );
}
```

```
    </h1>
  );
};
```

Eg : Conditional Rendering Using Boolean Prop

- Create a component called LoginStatus.
- Pass a prop called isLoggedIn (boolean).
- If isLoggedIn is true, display: "Welcome back!" Otherwise, display: "Please log in."

App.js

```
<LoginStatus isLoggedIn={true} />
<LoginStatus isLoggedIn={false} />
```

LoginStatus.js

```
function LoginStatus(props) {
  return <h2>{props.isLoggedIn ? "Welcome back!" : "Please log in."}</h2>;
}
```

Eg: Passing a Function as a Prop

- Create a component called AlertButton.
- Pass a prop called onClickHandler, which is a function.
- Inside AlertButton, render a button. When clicked, call the passed function.

App.js

```
function App() {
  const showAlert = () => {
    alert("Button clicked!");
  };
};
```

```
<AlertButton onClickHandler={showAlert} />
```

AlertButton.js

```
function AlertButton(props) {
  return <button onClick={props.onClickHandler}>Click Me</button>;
}
```

Eg: Using children Prop

- Create a component called Box.
- It should render a <div> with some styles and display whatever content is passed between <Box>...</Box> using props.children.

App.js

```
<Box>
  <p>This is inside the box!</p>
</Box>
```

`</Box>`

Box.js

```
function Box(props) {  
  const style = {border: '2px solid gray',padding: '20px',borderRadius: '5px',margin: '10px 0' };  
  return <div style={style}>{props.children}</div>;  
}
```

Eg: Dynamic Class Name Based on Prop

- Create a component called StatusTag.
- Accept a prop called status ("active" or "inactive").
- Display a colored tag:
 Green for "active"
 Red for "inactive"

App.js

```
<StatusTag status="active" />  
<StatusTag status="inactive" />
```

StatusTag.js

```
function StatusTag(props) {  
  const styles = {  
    active: { color: 'white', backgroundColor: 'green', padding: '5px 10px', borderRadius: '4px' },  
    inactive: { color: 'white', backgroundColor: 'red', padding: '5px 10px', borderRadius: '4px' }  
  };  
  return (  
    <span style={props.status === 'active' ? styles.active : styles.inactive}>  
      {props.status}  
    </span>  
  );  
};
```

Eg: Display Product Info Using Object Props

- Create a component called ProductCard.
- Pass a prop named product, which is an object with name, price, and inStock.
- Display product name, price, and a message:
 "In Stock" if inStock is true
 "Out of Stock" if false

App.js

```
const product = {name: "Laptop", price: "$999",inStock: true };  
<ProductCard product={product} />
```

ProductCard.js

```
function ProductCard(props) {  
  const { name, price, inStock } = props.product;  
  return (  
    <div style={{ border: '1px solid #ccc', padding: '15px', width: '250px' }}>  
      <h3>{name}</h3>
```

```
<p>Price: {price}</p>
<p>{inStock ? "In Stock" : "Out of Stock"}</p>
</div> );}
```

Assessment:

Passing Multiple Props

- Create a component named UserInfo.
- Pass three props to it from App: name, age, and city.
- Display them in a formatted sentence like:
"My name is [name], I am [age] years old, and I live in [city]."

Props with a Button Action Text

- Create a component called ActionButton.
- Pass one prop called label from the parent component.
- Render a button with the text from label prop.

Render a List Using Props

- Create a component called TodoList.
- Pass a prop called items which is an array of strings.
- Render each item in an unordered list ().

React Props Exercises

1. Create a component that receives a name as a prop and displays "Hello, [name]".
2. Create a component that receives an age number as a prop and displays a message using it.
3. Make a component that takes a boolean prop isLoggedIn and displays "Welcome!" or "Please log in".
4. Pass an object (with name and email) as a prop and display both values in a child component.
5. Create a list of items (array) in the parent and pass each item to a child <Item /> component using props.
6. Create a button in a child component. Pass a function from the parent component to handle button click.
7. Create a component that uses props.children to display custom inner content inside a styled card.
8. Use default props to set a default value for a name prop if it's not passed.
9. Pass two props: title and price, and display them in a ProductCard component.
10. Use destructuring to extract props in the function parameter of a component.
11. Create a component <Profile /> that accepts name, age, and location and renders a user profile layout.
12. Create a component that accepts a prop imageUrl and displays an image using it.
13. Create a button component that accepts label and onClick props and performs an action.
14. Pass a prop to style a component conditionally (e.g., isActive to change color).
15. Create a component that renders different JSX based on a prop userRole (e.g., Admin/User).
16. Pass an array of student names as a prop and render each in a child component.
17. Pass a number prop and create a component that renders a specific number of stars/icons based on it.
18. Pass multiple components or JSX as children using props.children and render them inside a layout component.

React useState Hook

- useState is a React Hook that lets you add state to functional components.
- Used to store values that **change over time** and affect how the component renders.
- It allows a component to remember values between renders.
- Importing useState

```
import { useState } from 'react';
```
- Syntax

```
const [state, setState] = useState(initialValue); //const[variable,setVariable]=useState(init_value);
```

state	The current state value
set State	Function to update the state
initialValue	The starting/default value for the state
- It can hold any data type

```
useState(0); // number           useState([]); // array
useState("Hello"); // string      useState({ name: "John" }); // object
useState(true); // boolean
```
- Never update state directly(to change usestate hook value)

```
count = count + 1; → Wrong(don't use assignment operator)
setCount(count + 1); → Right(use function created )
```

Eg: Basic Counter

```
import React, { useState } from 'react';
function Counter() {
  const [count, setCount] = useState(0);
  function show()
  {
    setCount(count+1)
  }
  return (
    <>
      <p>Count: {count}</p>
      <button onClick={show}>Increment</button>
    </>
  );
}
```

Eg: Basic Counter(alternate)

```
import React, { useState } from 'react';
function Counter() {
  const [count, setCount] = useState(0);
  return (
    <>
      <p>Count: {count}</p>
```

```

    <button onClick={() => setCount(count + 1)}>Add</button>
  </>
);
}

```

Eg: Toggle Button Text

```

const [isOn, setIsOn] = useState(true);
<button onClick={() => setIsOn(!isOn)}> {isOn ? 'ON' : 'OFF'} </button>

```

Eg: Change Input Text

```

const [text, setText] = useState("");
<input type="text" value={text} onChange={(e) => setText(e.target.value)} />

```

Eg: Show/Hide Message

```

const [visible, setVisible] = useState(true);
<button onClick={() => setVisible(!visible)}>Toggle</button>
{visible && <p>Hello there!</p>}

```

Eg: Counter with Reset

```

const [count, setCount] = useState(0);
<p>{count}</p>
<button onClick={() => setCount(count + 1)}>+</button>
<button onClick={() => setCount(0)}>Reset</button>

```

Eg: Array State

```

const [items, setItems] = useState([]);

const addItem = () => {
  setItems([...items, `Item ${items.length + 1}`]);
};

<button onClick={addItem}>Add Item</button>
<ul>
  {items.map((item, i) => <li key={i}>{item}</li>)}
</ul>

```

Eg: Object State

```

const [profile, setProfile] = useState({ name: "", age: "" });
<input value={profile.name} onChange={(e) => setProfile({ ...profile, name: e.target.value })} />
<input value={profile.age} onChange={(e) => setProfile({ ...profile, age: e.target.value })} />
<p>{profile.name} - {profile.age}</p>

```

Eg: Form with Multiple Fields

```
const [form, setForm] = useState({ username: "", password: "" });
```

```
const handleChange = (e) => {  
  setForm({ ...form, [e.target.name]: e.target.value });  
};
```

```
<input name="username" onChange={handleChange} />  
<input name="password" type="password" onChange={handleChange} />  
<p>{form.username}, {form.password}</p>
```

Eg: Conditional Color Change

```
const [color, setColor] = useState("blue");
```

```
<div style={{ backgroundColor: color, height: "50px", width: "50px" }}></div>  
<button onClick={() => setColor("green")}>Change to Green</button>
```

Eg: Track Clicks Count

```
const [clicks, setClicks] = useState(0);
```

```
<p>Clicked {clicks} times</p>  
<button onClick={() => setClicks(clicks + 1)}>Click</button>
```

Assessments:

1. Create a counter with three buttons: +1, -1, and Reset.
2. Create a form with an input field. Show the typed text below the input.
3. Build a toggle button that changes background color when clicked.
4. Create a login form with two inputs: username and password. Display the entered values below.
5. Create a component that adds random numbers to an array and displays them in a list.
6. Track how many times a user has clicked a button. Show a message after 5 clicks.
7. Create a form with two fields: name and email. On submit, clear the form.
8. Create a component that shows or hides a message using a toggle button.
9. Use useState to manage a theme (light/dark mode) and toggle between themes.
10. Build a mini task list where you can add tasks one by one and display them.
11. Create a child component that sends a message to the parent via a callback function prop.
12. Pass a form input value and onChange handler to a component using props.
- 13.

UseRefHook

To shift focus to another control

Item.js:

```
import {useRef} from 'react'

const inputRef=useRef()
:
:
<input ref={inputRef}
:
:
<button onclick={()=>inputRef.current.focus()}
```

UseEffectHook:

If no dependency , executes when page load
 when changes made in components(like key stroke,...) ie whenever the component renders this hook executes and this leads to more process to server(slowdown), so use [](empty array) if no dependencies

Eg:

App.js

```
Import {useEffect} from 'react'
useEffect(()=>{
  console.log("Rendering...")
})
```

Eg:

App.js

```
Import {useEffect} from 'react'
useEffect(()=>{
  console.log("Load")
},[])
```

Eg:

App.js

```
Import {useEffect} from 'react'
useEffect(()=>{
  console.log("To do List Changed")
},[count])
```

Eg: Order of execution

App.js

```
Import {useEffect} from 'react'
```

```

Console.log("Before...")
useEffect(()=>{
  console.log("Rendering...")
},[items])
Console.log("After...")

```

Referential Equality:-

Whenever the value of state changed re renders accrued
 Whenever re render accrued all functions are recreated again is
 known as "referential equality"

UseCallback Hook :

reduce the number of recreation of a function
 used to memorize callback functions and returns a memorized function.

Two arguments

A function that returns the memorized function

An array of dependencies

The memorized function is only re-created when one of the dependencies changes

Eg:

App.js

```

const [count,setCount]=useState(0);
const handleClick=()=>{
  setCount(count+1)
}
const handleNothing=()=>{
  console.log("Nothing...");
}
return(
  <p>{count}</p>
  <button onclick={handleClick}>Click</button>
)

```

Here, I need handleClick function recreated only when there is a change in count state

So convert this function into callback function

```

const handleClick=useCallback(()=>{
  setCount(count+1)
},[count])

```

Note:

We can't use setcount(any setstate) in useeffect (if used it is an infinite loop)

Eg:

```
const handleClick=useCallback(()=>{
    setCount(count+1)
},[])
```

If there is no dependency and just an empty array the count value is increased only once. We cant increase further. This is the power of usecallback

UseMemo Hook:

Used to memorize expensive computations and returns memorized value

Two arguments

A function that returns the value to be memorized

An array of dependencies

The memorized value is only re-computed when one of the dependencies changes

Eg:

```
const [count,setCount]=useState(0);
const expensiveValue==useMemo(()=>{
    let result=0
    for(let index=0;index<1000000000;index++)
    {
        result+=index
    }
    return result
},[count])
const handleClick=()=>{
    setCount(count+1)
}
const handleNothing=()=>{
    console.log("Nothing...");
}
return(
<p>{expensiveValue}</p>
<p>{count}</p>
<button onclick={handlClick}>Click</button>
)
```

Click button , notice that it get few seconds to increase count coz of useMemo

If there is no dependency in use memo

```
const expensiveValue==useMemo(()=>{
  let result=0
  for(let index=0;index<1000000000;index++)
  {
    result+=index
  }
  return result
},[])
```

Its get executed only once

Router:-

```
npm install react-router-dom
```

API:

Application Programming Interface (software to software interface)

They allow different applications to talk to each other and exchange information and functionality

API Call:

Process of a client application submitting a request to an API and that API retrieving the requested data from the external server or program and delivering it back to the client

API Eg 1:

Goto api.pheny.io/free-api/

Random joke API:

https://official-joke-api.appspot.com/random_joke

app.js

```
const url="https://official-joke-api.appspot.com/random_joke"
const apiFunction={()=>{
  const apiData=fetch(url)
  const data=apiData.json();
  console.log(data)
}}
```

```
}  
apiFunction()
```

JavaScript doesn't wait until fetch function completed(gets random joke). So, use async function with await (as immediate function)

```
const apiFunction=async()=>{  
    const apiData=await fetch(url,{method: "GET"})  
  
    const data=await apiData.json();  
    console.log(data)  
}
```

Home.js

```
const [apistate,setApistate]=useState("")  
const apiFunction=async()=>{  
    const apiData=await fetch(url)  
    const data=await apiData.json();  
    console.log(data)  
    setApistate(data)  
}  
apiFunction()
```

<p>{apistate.setup}</p>

Eg:On button click

Home.js

```
const url=process.env.REACT_APP_API_URL  
const [apistate,setApistate]=useState("")  
const apiFunction=async()=>{  
    const apiData=await fetch(url,{method:"GET"})  
    const data=await apiData.json();  
    console.log(data)  
    setApistate(data)  
}  
<p>{apistate.setup}</p>  
<p>{apistate.id}</p>  
  
<button onclick={apiFunction}>Click</button>
```